

Flutter と Dart プログラミング

Flutter は、美しいユーザーインターフェース (UI) を持つアプリを、Android と iOS の両方で動作するように簡単に開発できるフレームワーク。

Flutter の裏側には Dart というプログラミング言語が使われている。

Dart はシンプルで高速、そして初心者にも学びやすいように設計されている。

Dart プログラミングの基本概念

1. Dart 変数

- 情報 (数字やテキスト) を保存するために使う。
- `var`, `int`, `double`, `String` などの型で宣言できる。

```
var name = 'Alice'; // A string variable
int age = 30;       // An integer variable
```

2. Dart の演算子

- 演算子は、変数や値に対して加算、減算、比較、代入などの操作を行うために使用される。
 - ・ より多くの演算子や使い方については、[こちら](#)をご覧ください。

```
int sum = 5 + 3; // Addition
bool isEqual = (5 == 5); // Comparison
```

3. Dart のキーワード

- キーワードとは、`if`, `for`, `class`, `void` など、Dart であらかじめ意味が定義された特別な単語のこと。
- キーワードは変数名などには使用できません。
- より多くのキーワードについては、[こちら](#)をご覧ください。

4. Dart 内蔵タイプ

- Dart にはさまざまなデータを扱うための組み込み型が用意されている：

◇ 数値 ([Numbers](#)) : `int`, `double`

`int`: 整数 (小数なし) & `double`: 浮動小数点数 (小数あり)

```
int x = 10;
double y = 3.14;
```

◇ 文字列 ([Strings](#)) : `String`

テキストを表すために使用する。

```
String greeting = 'Hello, World!';
```

- ◇ 真偽値 ([Booleans](#)) : bool
true または false の値を持つ.

```
bool isActive = true;
```

- ◇ レコード ([Records](#)) : (値 1, 値 2)
複数の値を 1 つにまとめる簡単な方法.

```
var record = (10, 'hello');
```

- ◇ 関数 ([Functions](#)) : Function
関数は処理を行い, 結果を返すことができる.

```
int add(int a, int b) {  
    return a + b;  
}
```

- ◇ リスト ([Lists](#)/ 配列) : List
複数の値を順序付きで保存する.

```
List<String> colors = ['red', 'blue', 'green'];
```

join() メソッドを使うと, リスト内の要素を 1 つの文字列に結合できる.

```
List<String> fruits = ['Apple', 'Banana', 'Cherry'];  
print(fruits.join(', ')); // Output: "Apple, Banana,  
Cherry"
```

- ◇ セット ([Sets](#)) : Set
重複のない値の集まる.

```
Set<int> uniqueNumbers = {1, 2, 3};
```

- ◇ マップ ([Maps](#)) : Map
キーと値のペアを保存する.

```
Map<String, int> scores = {'Alice': 90, 'Bob': 85};
```

5. Dart Generic

- ジェネリクスを使うことで, 異なるデータ型に対応できる柔軟なコードが書ける.
- 例えば, 整数だけを保持するリストを定義するには:

```
List<int> numbers = [1, 2, 3];
```

6. Dart のエラー処理

- プログラム実行中に問題が起きたときに,それを管理・処理する方法.
- Dartでは try,catch,finally を使う.

```
try {
  int result = 10 ~/ 0;
} catch (e) {
  print('Cannot divide by zero: $e');
}
```

7. Dart OOP

- Dart は OOP に対応しており,クラスを使ってオブジェクトを定義できる.
- オブジェクトはプロパティ (変数) とメソッド (関数) を持つ.

```
class Car {
  String make;
  String model;

  Car(this.make, this.model);

  void displayInfo() {
    print('Car: $make $model');
  }
}

void main() {
  var myCar = Car('Toyota', 'Corolla');
  myCar.displayInfo();
}
```

列挙型 (Enum)

- Enum (列挙型) は,特定の固定された値を表す特別な型.
- 特定の値しか許可したくないときに便利.

```
enum CarType { sedan, suv, truck }

void main() {
  CarType myCarType = CarType.sedan;

  if (myCarType == CarType.sedan) {
    print('Your car is a sedan.');

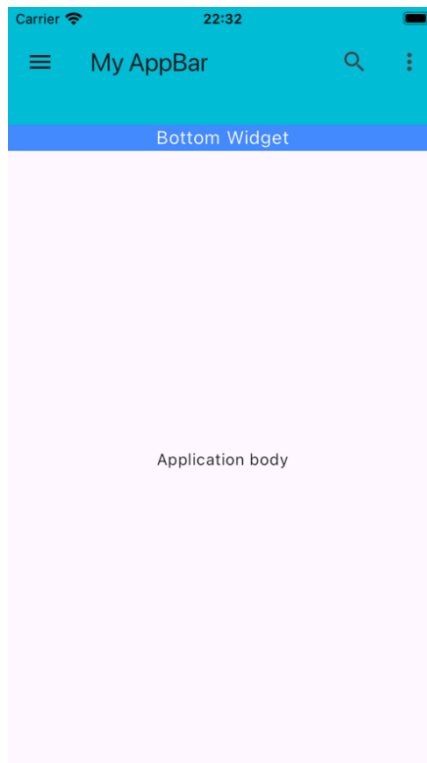
```

8. Dart Null Safety

- 変数が「null」になることによるエラーを防ぐ仕組み.
- Dartでは変数が null を取れるかどうかを明示する必要がある.

```
String? nullableString; // This can be null
```

```
String nonNullableString = 'Hello'; // This cannot be  
null
```



d. AppBar を使うべき場面

- AppBar がよく使われる場面:
 - ◇ アプリの統一感のあるトップバーが必要なとき.
 - ◇ タイトルやナビゲーションボタンを追加したいとき.
 - ◇ 検索,設定,その他のオプションを提供したいとき.
 - ◇ タブや追加コンテンツを AppBar の下に表示したいとき.

Flutter の Text ウィジェット

a. Text ウィジェットとは？

- Flutter アプリでテキストを表示するウィジェット.
- プレーンテキストとスタイル付きテキストの両方をサポート.
- 静的メッセージ,動的更新,レスポンスデザインに対応可能.

b. 主なプロパティ

- 主なプロパティ：

プロパティ	説明	使用例
data	表示するテキスト（必須）.	'Hello, Flutter!'
style	テキストの見た目を変更.	TextStyle(fontSize: 18, color: Colors.blue)

textAlign	テキストの配置(left, center, right).	TextAlign.center
maxLines	最大行数を制限.	maxLines: 2
overflow	テキストのオーバーフロー処理 (ellipsis, fade).	TextOverflow.ellipsis
softWrap	テキストの折り返し設定.	softWrap: true

c. 例：FlutterでのTextスタイルの設定

■ 例のFlutterコード：

```
Text(
  'Welcome to Flutter!', // The text to display
  style: TextStyle(
    fontSize: 20,          // Sets the font size
    color: Colors.green,   // Text color
    fontWeight: FontWeight.bold, // Makes the text bold
  ),
  textAlign: TextAlign.center, // Centers the text
  maxLines: 2,                // Limits to 2 lines
  overflow: TextOverflow.ellipsis, // Adds "..." if the
text is too long
)
```

■ 動作説明:

- ◇ 'Welcome to Flutter!' → 表示するテキスト.
- ◇ style → サイズ,色,太さなどを指定.
- ◇ textAlign: TextAlign.center → テキストを中央揃え.
- ◇ maxLines: 2 → 最大2行までに制限.
- ◇ overflow: TextOverflow.ellipsis → 長すぎる場合“…”を表示.

d. なぜTextウィジェットを使うのか？

■ Textウィジェットを使う理由:

- ◇ 静的・動的テキストの表示が簡単.
- ◇ 整列・折り返し・オーバーフロー処理が可能.
- ◇ どんなレイアウトやUIデザインにも適用できる.

FlutterのContainerウィジェット

a. Containerウィジェットとは？

- スタイル,位置,レイアウトを設定できるカスタマイズ可能な「ボックス」.
- 他のウィジェットをラップし,間隔,色,装飾を適用するのによく使われる.
- Flutterで最も柔軟で一般的に使用されるウィジェットの1つ.